MODUL PRAKTIKUM LARAVEL
STATELESS SHOPPING CART

1. Tujuan Praktikum

- Melanjutkan kasus toko online yang sama: Buku Tulis, Pulpen, dan Penggaris.
- Mengimplementasikan operasi keranjang belanja menggunakan Laravel tanpa menyimpan keranjang ke cookie atau session.
- Memahami bahwa setiap request harus membawa data keranjang yang diperlukan.
- Menguji perilaku request melalui browser dan Postman.

2. Aturan Bisnis

Gunakan katalog yang sama dengan modul awal !

ID	Produk	Harga
1	Buku Tulis	Rp5.000
2	Pulpen	Rp3.000
3	Penggaris	Rp4.000

3. Tahap 1 — Menyiapkan Proyek Laravel

Jika belum mempunyai proyek, silahkan menyiapkan dengan instruksi berikut.

```
composer create-project laravel/laravel stateless-shop
cd stateless-shop
php artisan serve
```

Jika proyek Laravel sudah tersedia, gunakan proyek tersebut. Database tidak diperlukan karena katalog produk dibuat sebagai array untuk menyamakan contoh pada modul awal.

4. Tahap 2 — Membuat Controller

Jalankan: php artisan make:controller CartController. Buka app/Http/Controllers/CartController.php dan gunakan kode berikut.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class CartController extends Controller
{
    private array $products = [
        1 => ['name' => 'Buku Tulis', 'price' => 5000],
        2 => ['name' => 'Pulpen', 'price' => 3000],
```

```
        3 => ['name' => 'Penggaris', 'price' => 4000],
    ];

    public function index()
    {
        return view('shop');
    }

    public function add(Request $request)
    {
        $data = $request->validate([
            'cart' => ['present', 'array'],
            'cart.*' => ['integer', 'min:1', 'max:99'],
            'product_id' => ['required', 'integer'],
        ]);

        $cart = $data['cart'];
        $id = (int) $data['product_id'];

        abort_unless(isset($this->products[$id]), 404);

        $cart[$id] = min(99, ($cart[$id] ?? 0) + 1);

        return response()->json($this->calculate($cart));
    }

    public function update(Request $request)
    {
        $data = $request->validate([
            'cart' => ['present', 'array'],
            'cart.*' => ['integer', 'min:1', 'max:99'],
        ]);

        return response()->json($this->calculate($data['cart']));
    }

    private function calculate(array $cart): array
    {
        $items = [];
        $total = 0;

        foreach ($cart as $id => $quantity) {
            if (!isset($this->products[$id])) {
                continue;
            }

            $product = $this->products[$id];
            $subtotal = $product['price'] * $quantity;

            $items[] = [
                'id' => (int) $id,
```

```
                'name' => $product['name'],
                'price' => $product['price'],
                'quantity' => $quantity,
                'subtotal' => $subtotal,
            ];

            $total += $subtotal;
        }

        return ['items' => $items, 'total' => $total];
    }
}
```

Perhatikan: `$data['cart']` dibaca dari payload request. Controller tidak memanggil session() dan tidak menyimpan keranjang ke cookie.

5. Tahap 3 — Mengatur Route

Tambahkan route berikut ke routes/web.php. Pertahankan route lain yang sudah ada.

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\CartController;

Route::get('/', [CartController::class, 'index']);
Route::post('/cart/add', [CartController::class, 'add'])-
>name('cart.add');
Route::post('/cart/update', [CartController::class, 'update'])-
>name('cart.update');
```

Periksa route dengan php artisan route:list. Route tetap berada di web.php agar dapat menggunakan Blade dan CSRF.

6. Tahap 4 — Membuat Tampilan Toko

Buat resources/views/shop.blade.php dengan kode berikut atau menggunakan *source code* yang sudah ada.

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="csrf-token" content="{{ csrf_token() }}">
    <title>Stateless Shopping Cart</title>
</head>
<body>
    <h1>Toko Online</h1>
    <h2>Daftar Produk</h2>
    <button onclick="add(1)">Buku Tulis - Rp5.000</button>
```

```
    <button onclick="add(2)">Pulpen - Rp3.000</button>
    <button onclick="add(3)">Penggaris - Rp4.000</button>

    <h2>Keranjang Belanja</h2>
    <div id="items">Keranjang kosong.</div>
    <h3 id="total">Total: Rp0</h3>
    <button onclick="recalculate()">Hitung Ulang</button>
    <button onclick="clearCart()">Kosongkan Keranjang</button>
    <h3>Respons Laravel</h3>
    <pre id="response">Belum ada request.</pre>

<script>
let cart = {};

async function post(url, payload) {
    const response = await fetch(url, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'Accept': 'application/json',
            'X-CSRF-TOKEN': document.querySelector(
                'meta[name="csrf-token"]').content
        },
        body: JSON.stringify(payload)
    });

    const data = await response.json();
    if (!response.ok) throw new Error(JSON.stringify(data));
    return data;
}

async function add(id) {
    try {
        const data = await post('/cart/add', {
            cart: cart,
            product_id: id
        });

        cart = Object.fromEntries(
            data.items.map(item => [item.id, item.quantity])
        );
        render(data);
    } catch (error) {
        alert(error.message);
    }
}

async function recalculate() {
    if (Object.keys(cart).length === 0) {
        alert('Keranjang kosong.');
        return;
```

```
    }

    try {
        render(await post('/cart/update', {cart: cart}));
    } catch (error) {
        alert(error.message);
    }
}

function render(data) {
    const container = document.getElementById('items');
    container.replaceChildren();

    if (data.items.length === 0) {
        container.textContent = 'Keranjang kosong.';
    }

    data.items.forEach(item => {
        const row = document.createElement('p');
        row.textContent = `${item.name} x ${item.quantity} = Rp${
            item.subtotal.toLocaleString('id-ID')}`;
        container.appendChild(row);
    });

    document.getElementById('total').textContent =
        'Total: Rp' + data.total.toLocaleString('id-ID');
    document.getElementById('response').textContent =
        JSON.stringify(data, null, 2);
}

function clearCart() {
    cart = {};
    render({items: [], total: 0});
}
</script>
</body>
</html>
```

7. Tahap 5 — Menjalankan Aplikasi

Jalankan php artisan optimize:clear lalu php artisan serve. Buka http://127.0.0.1:8000.

Klik Buku Tulis dua kali, kemudian Pulpen satu kali. Hasil yang diharapkan:

Produk	Jumlah	Subtotal
Buku Tulis	2	Rp10.000
Pulpen	1	Rp3.000
TOTAL		Rp13.000

8. Tahap 6 — Mengamati Operasi Stateless

1. Buka DevTools → Network. Klik tombol Tambah dan buka request cart/add. Amati Request Payload dan Response.
2. Payload berisi cart sebelumnya dan product_id. Response berisi hasil setelah penambahan.
3. Refresh halaman. Keranjang akan kosong karena variabel JavaScript cart belum disimpan ke localStorage atau cookie.
4. Kirim payload yang sama berulang kali melalui Postman. Responsnya sama jika payload yang dikirim sama.

Contoh payload POST /cart/add:

```
{
  "cart": {"1": 2},
  "product_id": 2
}
```

Respons yang diharapkan: dua Buku Tulis dan satu Pulpen, total Rp13.000. Karena route berada di web.php, Postman perlu token CSRF yang valid.

9. Diskusi dan Evaluasi

1. Di baris kode mana browser mengirim state keranjang ke Laravel?
2. Mengapa controller tidak perlu membaca session untuk menghitung keranjang?
3. Apa yang terjadi jika request dikirim ulang dengan payload yang sama?
4. Mengapa keranjang kosong setelah halaman di-refresh?
5. Apakah penggunaan middleware session Laravel berarti keranjang ini disimpan dalam session? Jelaskan.
6. Apa risiko keamanan jika server mempercayai harga yang dikirim browser?

10. *Lesson-learned*

Tuliskan lesson-learned yang Anda peroleh setelah memahami konsep stateful dan stateless!

.